PathBonnet — Phase 2 Project Report

A Strong Dense Baseline for H&E Nuclei Segmentation: DeepLabV3+ on the NuInsSeg Benchmark

Mithra K
Independent Researcher, Scottsdale, Arizona, USA
October 2026

1 Summary

Phase 2 of the PathBonnet project set out to establish how far a strong, modern dense segmentation network can go on nuclei segmentation in H&E-stained histology images, as a reference point for the sparse-computation approach studied in Phase 1. A DeepLabV3+ network with a ResNet-101 encoder (45.7 million parameters) was trained on the NuInsSeg benchmark (665 images) using the dataset authors' five-fold split scheme. On the held-out test fold the model reached a foreground Dice of 0.837 at 58.2 ms per image, with the validation and test scores within 0.002 of each other. The result sits above the best figure reported in the dataset's own published benchmark (0.814, a five-fold mean), although our number comes from a single fold, so the comparison is indicative rather than like-for-like. Phase 2 therefore gives the project a credible dense reference point. It does not disturb the central finding of Phase 1 — that sparse inference offers no speed advantage at patch scale — because that finding rests on tissue occupancy, a property of the images rather than of any particular network.

2 Background and rationale

2.1 Where Phase 1 ended

PathBonnet began by adapting a sparse segmentation pipeline, originally developed for CT bone segmentation (Bonnet), to the separation of cell nuclei in H&E-stained tissue sections. A stain-deconvolution prior derived from the haematoxylin channel marked the pixels worth computing on, and a sparse two-dimensional U-Net processed only those locations. The prior itself worked well — it captured 98% of all nuclear pixels — but the end-to-end comparison against a dense counterpart of the same network produced a clear negative result: the sparse models were five to six times slower than the dense baseline (59.6–75.6 ms per image against 12.2 ms), at equal or slightly lower accuracy, and the ordering repeated on an external dataset. The explanation is structural. Within a tissue patch, nuclei and their immediate surroundings occupy most of the field, so the prior removes only

about a third of the pixels; the bookkeeping that sparse computation carries then costs more than the arithmetic it saves.

2.2 Why a stronger dense baseline

The Phase 1 comparison was deliberately matched — sparse and dense versions of one small network (2.3 million parameters) — which is the right design for isolating the effect of sparsity, but it leaves a question open: how much of the remaining accuracy gap is simply model capacity? A fair map of the design space needs a reference point from a contemporary, high-capacity dense architecture. DeepLabV3+ was selected for that role. Its encoder–decoder design, with atrous spatial pyramid pooling gathering context at several scales at once, suits nuclei that vary in size and crowding, and it is a standard, well-tested choice in biomedical segmentation. Phase 2 also moves the evaluation to NuInsSeg, a larger and more varied benchmark than the Phase 1 sets, so that the reference point is measured where it will be used.

3 Dataset

NuInsSeg contains 665 H&E-stained images of 512 × 512 pixels, gathered from 31 human and mouse organs, with more than 30,000 manually annotated nuclei. Each image ships with a binary nuclei mask, an instance mask, and an ambiguous-area mask marking regions that even expert annotators could not label with confidence.

Curation. The public archive stores every image several times over, once inside each of several folder organisations (by organ, by mask type, and so on): 7,315 image files in total for 665 distinct images. Training on the archive as downloaded would place copies of the same tissue in both the training and evaluation partitions and silently inflate the scores. The loader therefore deduplicates by filename stem — 6,650 duplicate copies are discarded, keeping the canonical tissue-image copy of each — and the partition is made over the 665 unique stems, so that no tissue image can appear in two partitions.

Splits and labels. We follow the dataset authors' five-fold cross-validation scheme (random seed 19), using folds 0–2 for training (399 images), fold 3 for validation (133), and fold 4 as the held-out test set (133). The released masks are binary — nuclei against background — and carry no nuclear-type labels, so the task is two-class; assigning type labels the dataset does not contain would fabricate information. Pixels covered by the ambiguous-area masks are excluded from both the loss and the metrics (ignore index 255), since scoring a model against regions that human experts left undecided would add noise rather than signal.

4 Method

4.1 Network

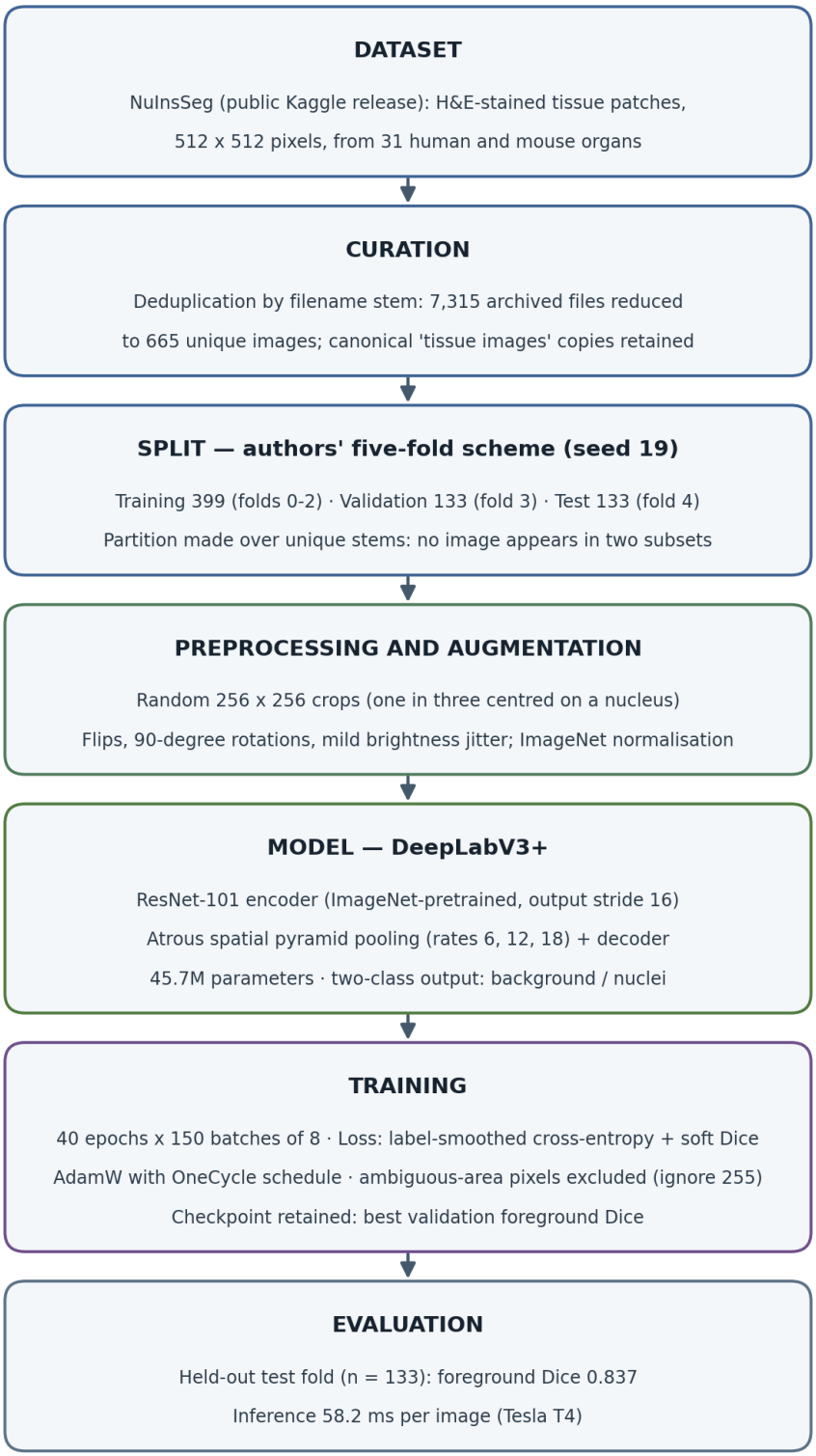


Figure 1 — The Phase 2 pipeline, from dataset curation and splitting through preprocessing, the DeepLabV3+ model and training, to evaluation on the held-out test fold.

The network is DeepLabV3+ as implemented in segmentation-models-pytorch (version 0.5.0). A ResNet-101 encoder, initialised with ImageNet weights, extracts features at output stride 16; the atrous spatial pyramid pooling module probes those features with dilation rates 6, 12 and 18 in parallel; and a compact decoder restores boundary detail before the final two-class prediction. The model has 45,673,506 parameters — roughly twenty times the Phase 1 U-Net. The input is the unmodified RGB patch: although the haematoxylin channel carries the nuclear signal, the pretrained encoder expects three channels, and the network is free to learn its own weighting of the stains. Whether restricting the input to the haematoxylin channel helps or hurts is a separate question, taken up in the planned ablation study (Section 7).

4.2 Training

Setting	Value
Architecture	DeepLabV3+ (segmentation-models-pytorch 0.5.0)
Encoder	ResNet-101, ImageNet-pretrained
Parameters	45,673,506
Input	512 × 512 RGB patch
Classes	2 (background, nuclei)
Loss	Cross-entropy (label smoothing 0.1) + soft Dice
Optimiser	AdamW (learning rate 2×10^{-3}, weight decay 10^{-4})
Schedule	OneCycleLR, peak learning rate 2×10^{-3}
Batch size	8
Training length	40 epochs × 150 batches
Augmentation	Horizontal/vertical flips, 90° rotations
Checkpoint selection	Best validation foreground Dice
Hardware	Kaggle, 2 × NVIDIA Tesla T4

Table 1 — Training configuration of the reported run.

The recipe deliberately mirrors the Phase 1 training protocol — the same loss family, optimiser style and schedule shape — changing the architecture and the dataset rather than the training method, so that differences in outcome are attributable to those two factors. The checkpoint retained for evaluation is the one with the highest validation foreground Dice.

4.3 Evaluation

Performance is reported as foreground Dice (the Dice coefficient over the nuclei class) on the held-out test fold, computed with ambiguous pixels excluded. Inference speed is the mean per-image time of the evaluation pass over the 133 test images on a Tesla T4, with the first image's warm-up cost set aside — the same timing convention used in Phase 1. Evaluation is a single run of the evaluation script over the saved checkpoint; the code path and the checkpoint are fixed artefacts, so the reported numbers are exactly reproducible.

5 Results

5.1 Training behaviour

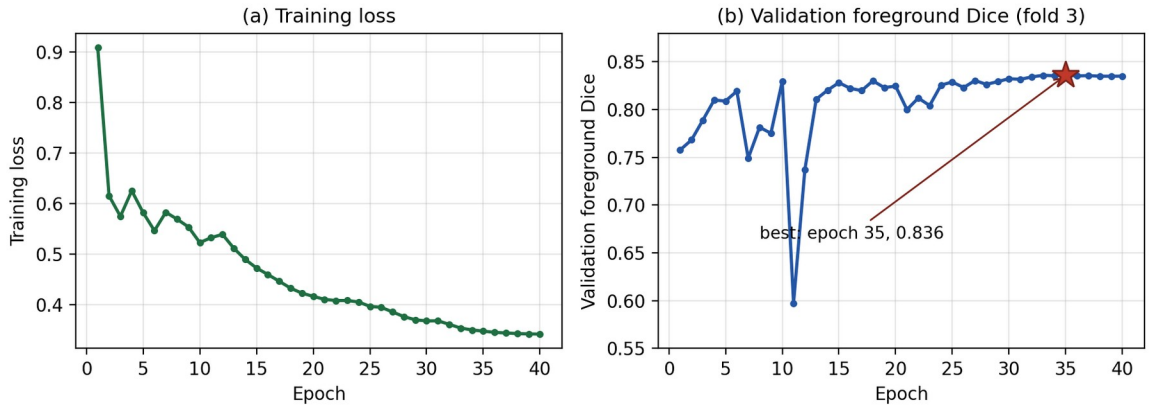


Figure 2 — Training loss (a) and validation foreground Dice on fold 3 (b) over 40 epochs, plotted from the run's training log. The star marks the retained checkpoint: epoch 35, validation Dice 0.836.

Training was uneventful in the ways that matter. The loss fell steadily from 0.91 in the first epoch to 0.34 by the last, and validation Dice climbed from 0.758 to a plateau: across the final ten epochs it stayed within 0.834–0.836, so the retained checkpoint (epoch 35, 0.836) sits on a flat maximum rather than on a lucky spike. Two early dips (epochs 7 and 11, the latter to 0.60) occurred while the OneCycle schedule held the learning rate near its peak and were recovered from immediately. The curve shows no sign of divergence or of overfitting within the 40-epoch budget — validation performance at the end of training is as good as at any earlier point.

5.2 Test performance

Measure	Value
Validation foreground Dice (fold 3, n = 133)	0.836
Test foreground Dice (fold 4, n = 133)	0.837
Inference time (Tesla T4)	58.2 ms per image

Table 2 — Headline results of the reported run.

On the held-out test fold the model scored a foreground Dice of 0.837 — within 0.002 of its validation score, as close an agreement as a single split can reasonably show, and a useful internal check that the test fold was neither unusually easy nor contaminated.

5.3 Context

Reference point	Foreground Dice
NuInsSeg published benchmark — best model, five-fold mean	0.814
Phase 2 DeepLabV3+ — this work, test fold 4 only	0.837
Phase 1 dense U-Net — PanNuke (different dataset, indicative)	0.797
Phase 1 sparse U-Net — PanNuke (different dataset, indicative)	0.781

Table 3 — The Phase 2 result in context.

Two comparisons frame the number. Against the dataset paper's own benchmark, in which the best of several models averaged a Dice of 0.814 across all five folds, our single-fold 0.837 sits above the published reference point. The comparison must be read with its caveat attached: a five-fold mean and a single fold are not the same statistic, and completing the remaining folds is the first item of future work. The instance-level metrics from the same benchmark (AJI 0.559, PQ 0.513) are not reported here, because a semantic model merges touching nuclei into a single region; measuring instance separation requires an instance-aware output head, also planned (Section 7). Against Phase 1, the comparison crosses datasets and is indicative only: the small dense U-Net reached 0.797 on PanNuke at 12.2 ms per image. DeepLabV3+ is the more accurate model by a clear margin, and also the slower one — capacity bought accuracy at roughly five times the latency.

6 Discussion

What Phase 2 establishes is a reference point the project did not have: on a current, varied nuclei benchmark, a standard high-capacity dense network, trained with the project's own protocol, reaches a foreground Dice of about 0.84, reproducibly from fixed artefacts. Any future PathBonnet variant — sparse, stain-ablated, distilled or instance-aware — can now be measured against a number that means something.

The result also sharpens, rather than softens, the Phase 1 conclusion. Sparsity was never competing against a weak baseline in the sense that matters: the argument was that at patch scale there is too little empty tissue for skipping computation to pay, and that argument is about the images, not the network. A sparse DeepLabV3+ would face the same occupancy arithmetic. If sparse processing is to win anywhere in this problem, it remains where Phase 1 pointed — at whole-slide scale, separating tissue from glass before patch-level analysis begins.

The honest limitations are threefold. First, a single fold: until the full five-fold cross-validation is run, the test figure carries fold-to-fold uncertainty that its agreement with validation can only partly reassure against. Second, the output is semantic, not instance-level; for diagnostic use, separating touching nuclei matters, and Dice alone cannot see the difference. Third, the timing was measured on shared cloud hardware (a Kaggle Tesla T4) and should be read as a like-for-like figure within this project rather than an absolute benchmark.

7 Next steps

Four extensions follow directly. (i) Complete the five-fold cross-validation of the present pipeline, so that the headline number becomes a mean with a spread, directly comparable to the published benchmark. (ii) Settle the input question with data: train matched models on RGB, on the haematoxylin channel alone, and on haematoxylin plus optical-density channels, with stain augmentation for robustness, and test on an external set (CPM-17). (iii) Add an instance-separation output — a horizontal/vertical distance-map head in the style of HoVer-Net — so that AJI and PQ can be reported alongside Dice. (iv) Distil the trained model into a compact student network, to find out how much of the 0.837 can be kept at something closer to the small U-Net's latency.

8 Reproducibility and artefacts

Training and evaluation code (train_deeplabv3plus.py, evaluate.py) accompanies this report; the model implementation is segmentation-models-pytorch 0.5.0. Runs were executed on Kaggle (2 × NVIDIA Tesla T4). The artefacts of the reported run are preserved: best.pt (the epoch-35 checkpoint), train_log.json (per-epoch loss and validation Dice) and eval_results.json (test metrics). The PathBonnet project repository is public at github.com/mithragoutham8-ai/pathbonnet.

References

1. Chen L-C, Zhu Y, Papandreou G, Schroff F, Adam H. Encoder-decoder with atrous separable convolution for semantic image segmentation. Proceedings of ECCV 2018. arXiv:1802.02611.

2. He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. Proceedings of CVPR 2016. arXiv:1512.03385.

3. Mahbod A, et al. NuInsSeg: a fully annotated dataset for nuclei instance segmentation in H&E-stained histological images. Scientific Data 11, 295 (2024). DOI: 10.1038/s41597-024-03743-3.

4. Zhu et al. Bonnet: sparse segmentation pipeline for CT bone segmentation (ISBI 2026). arXiv:2601.22576.

5. Gamper J, Alemi Koohbanani N, Benes K, Graham S, et al. PanNuke: an open pan-cancer histology dataset for nuclei instance segmentation and classification. In: European Congress on Digital Pathology (ECDP) 2019, LNCS 11844.

6. Ruifrok AC, Johnston DA. Quantification of histochemical staining by color deconvolution. Analytical and Quantitative Cytology and Histology 23(4): 291–299 (2001).

7. Loshchilov I, Hutter F. Decoupled weight decay regularization. Proceedings of ICLR 2019. arXiv:1711.05101.

8. Iakubovskii P. Segmentation Models Pytorch. GitHub, 2019. github.com/qubvel/segmentation_models.pytorch.